R demo and testing

Alfred

2026-01-20

```
setwd("C:/R-statistics-analysis")
```

Demo and testing of different things in R

This markdown file serves as an environment to review concepts and demo visualising of data using the R language and RStudio.

1. Visualising probability distributions

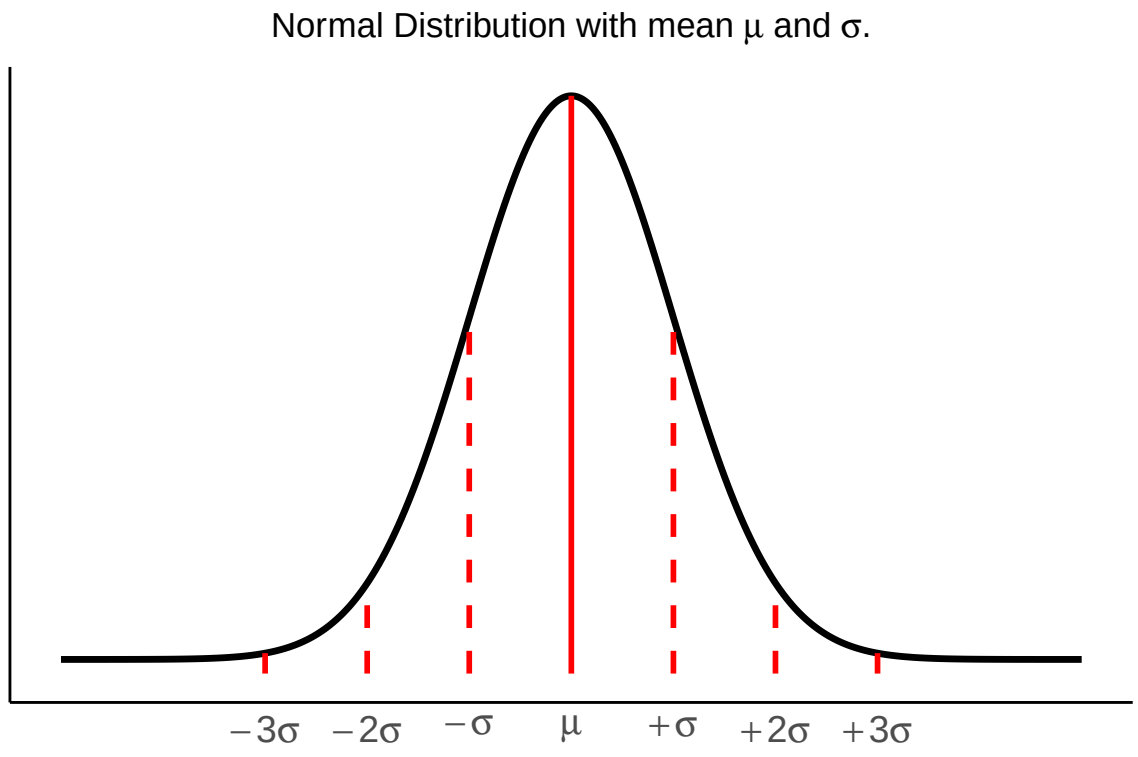
1.1 Normal distributions

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

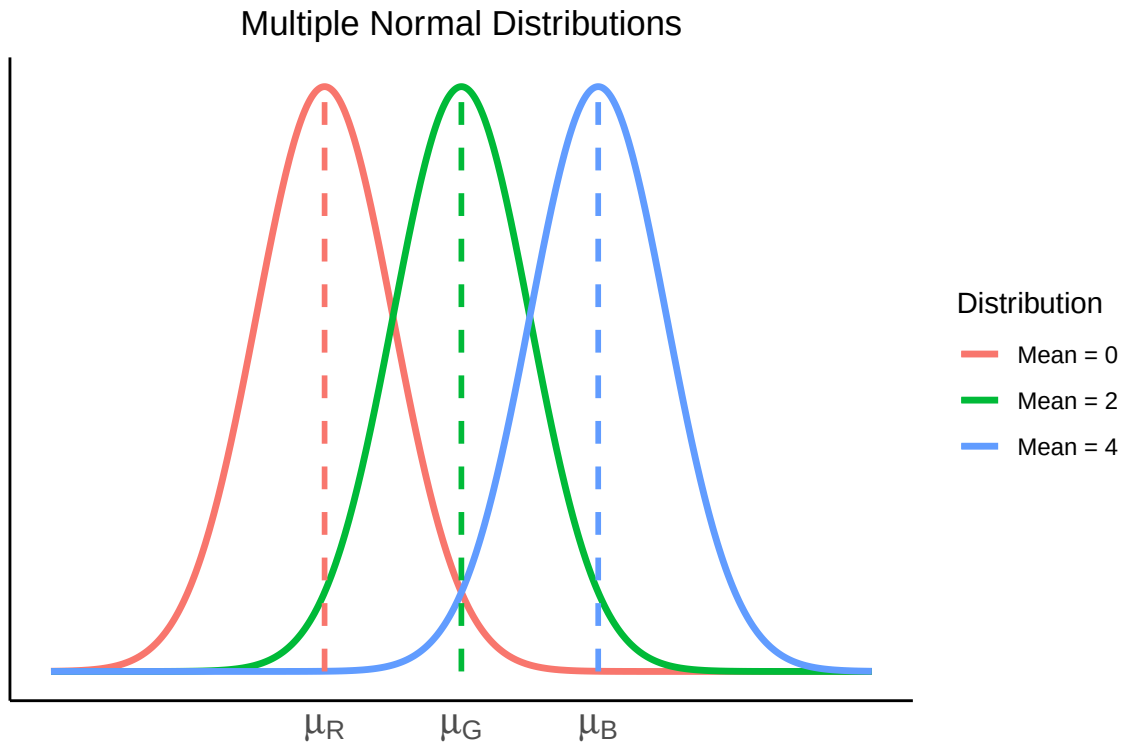
```
x <- seq(-5, 5, length.out = 1000)

df <- data.frame(
  x = x,
  density = dnorm(x, mean = 0, sd = 1)
)

# Positions for vertical lines
vline_positions <- c(-3, -2, -1, 0, 1, 2, 3)

# Data frame for vertical segments
vline_df <- data.frame(
  x = vline_positions,
  y_start = -0.01,
  y_end = dnorm(vline_positions, mean = 0, sd = 1)
)

ggplot(df, aes(x = x, y = density)) +
  geom_line(linewidth = 1.2, color = "black") +
  geom_segment(
    data = vline_df,
    aes(x = x, xend = x, y = y_start, yend = y_end),
    linetype = c(2, 2, 2, 1, 2, 2, 2),    # dashed for -1 and 1, solid for 0
    linewidth = 1,
    color = "red"
  ) +
```

```
  scale_x_continuous(
    breaks = c(-3, -2, -1, 0, 1, 2, 3),
    labels = expression(-3*sigma, -2*sigma, -sigma, mu, +sigma, +2*sigma, +3*sigma)
  ) +
  labs(
    title = expression("Normal Distribution with mean " * mu * " and " * sigma * "."),
    x = NULL,
    y = NULL
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
    axis.text.y = element_blank(),
    axis.text.x = element_text(size = 14)
  )
```




```
library(ggplot2)

x <- seq(-4, 8, length.out = 1000)

df <- data.frame(
  x = rep(x, 3),
  density = c(
    dnorm(x, mean = 0, sd = 1),
    dnorm(x, mean = 2, sd = 1),
    dnorm(x, mean = 4, sd = 1)
  ),
```

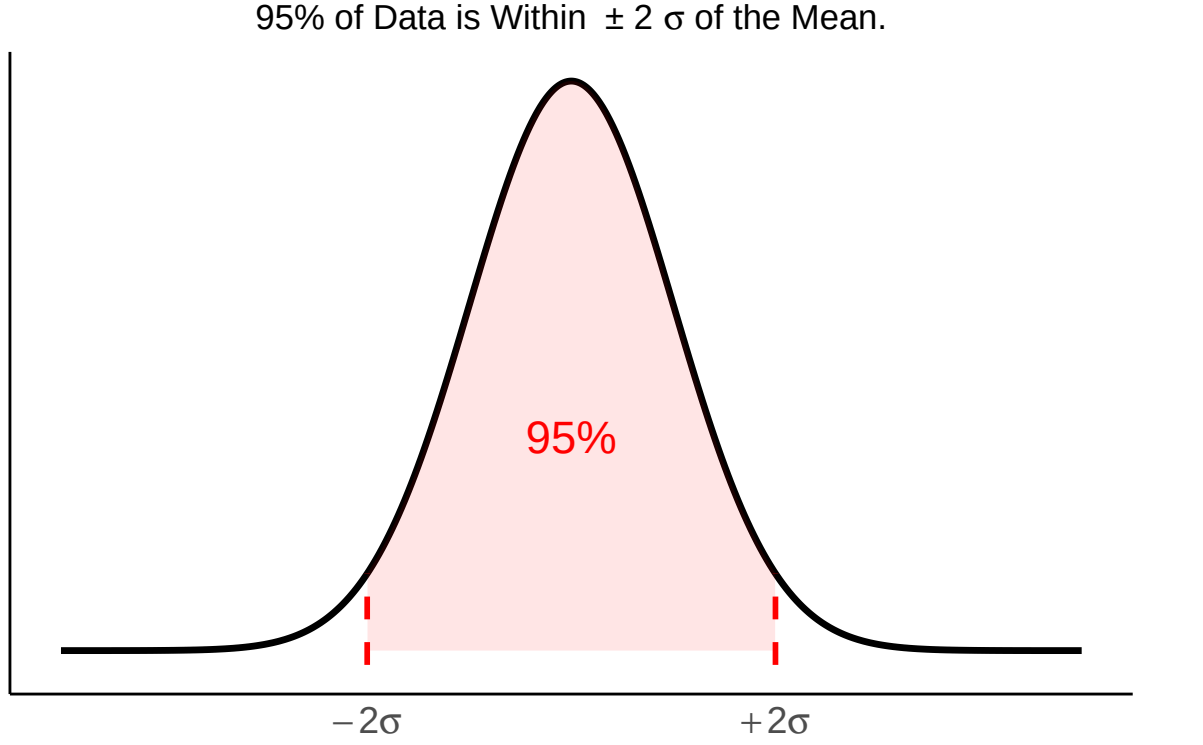
```
  distribution = factor(
    rep(c("Mean = 0", "Mean = 2", "Mean = 4"), each = length(x))
  )
)

# Data frame for vertical mean segments
vline_df <- data.frame(
  x = c(0, 2, 4),
  mean = c(0, 2,4),
  distribution = factor(
    expression("Mean = 0", "Mean = 2", "Mean = 4"),
    levels = levels(df$distribution)
  )
)

# Segment start/end
vline_df$y_start <- 0
vline_df$y_end <- dnorm(vline_df$x, mean = vline_df$mean, sd = 1)

ggplot(df, aes(x = x, y = density, color = distribution)) +
  geom_line(linewidth = 1.2) +
  geom_segment(
    data = vline_df,
    aes(
      x = x, xend = x,
      y = y_start, yend = y_end,
      color = distribution
    ),
    linewidth = 1,
    linetype = "dashed"
  ) +
  scale_x_continuous(
    breaks = c(0, 2, 4),
    labels = expression(mu[R], mu[G], mu[B])
  ) +
  labs(
    title = "Multiple Normal Distributions",
    x = NULL,
    y = NULL,
    color = "Distribution"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
    axis.text.y = element_blank(),
    axis.text.x = element_text(size = 14)
  )
```



1.2 Visualising probability as area under the curve

```
library(ggplot2)

x <- seq(-5, 5, length.out = 1000)

df <- data.frame(
  x = x,
  density = dnorm(x, mean = 0, sd = 1)
)

# Positions for vertical lines
vline_positions <- c(-2, 2)

# Data frame for vertical segments
vline_df <- data.frame(
  x = vline_positions,
  y_start = -0.01,
  y_end = dnorm(vline_positions, mean = 0, sd = 1)
)

ggplot(df, aes(x = x, y = density)) +
  geom_line(linewidth = 1.2, color = "black") +
  geom_ribbon(
    data = subset(df, x >= -2 & x <= 2),
    aes(x = x, ymin = 0, ymax = density),
    alpha = 0.1,
    inherit.aes = FALSE,
    fill = "red"
```

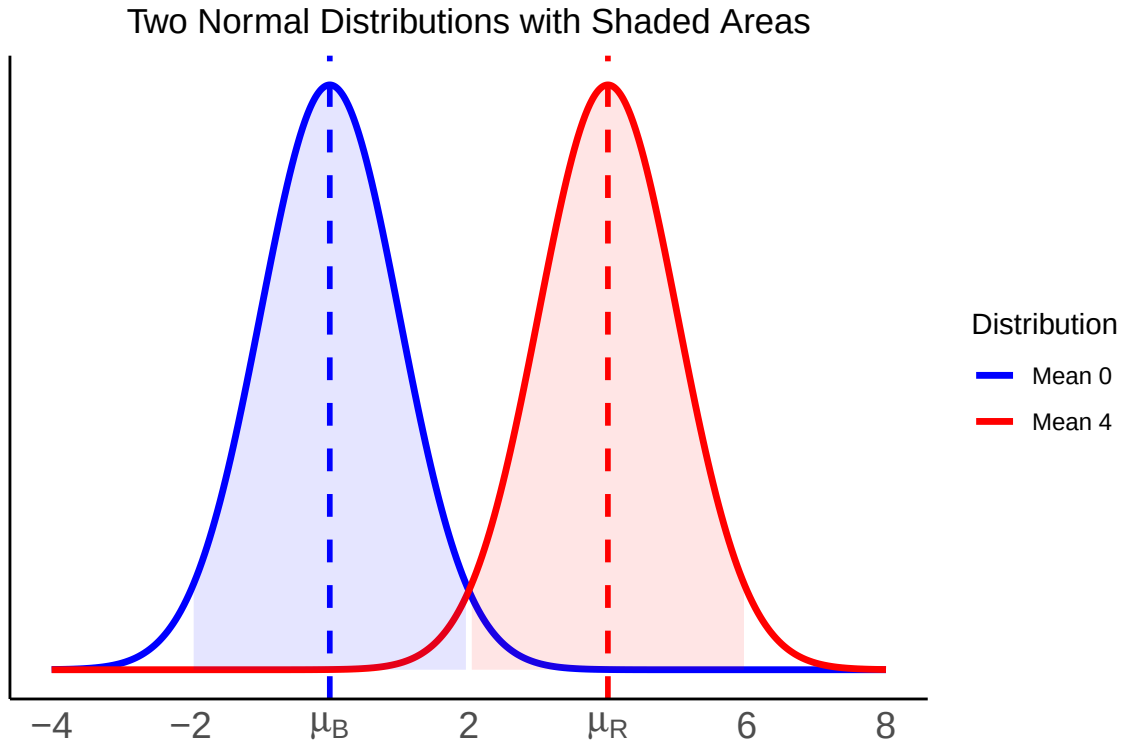
```
  ) +
  geom_segment(
    data = vline_df,
    aes(x = x, xend = x, y = y_start, yend = y_end),
    linetype = c(2, 2), # all dashed lines
    linewidth = 1,
    color = "red"
  ) +
  scale_x_continuous(
    breaks = c(-2, 2),
    labels = expression(-2*sigma, +2*sigma)
  ) +
  labs(
    title = expression("95% of Data is Within " * ~"\u00B1 2 " * sigma * " of the Mean."),
    x = NULL,
    y = NULL
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
    axis.text.y = element_blank(),
    axis.text.x = element_text(size = 14)
  ) +
  annotate("text", x = 0, y = 0.15, label = "95%", color = "red", size = 6)
```




```
library(ggplot2)
```

```r
x <- seq(-4, 8, length.out = 1000)

df <- data.frame(
  x = rep(x, 2),
  density = c(
    dnorm(x, mean = 0, sd = 1),
    dnorm(x, mean = 4, sd = 1)
  ),
  distribution = factor(rep(c("Mean 0", "Mean 4"), each = length(x)))
)

ggplot(df, aes(x = x, y = density, color = distribution, fill = distribution)) +
  geom_line(linewidth = 1.2) +
  geom_ribbon(
    data = subset(df, distribution == "Mean 0" & x >= -1.96 & x <= 1.96),
    aes(x = x, ymin = 0, ymax = density),
    alpha = 0.1,
    inherit.aes = FALSE,
    fill = "blue"
  ) +
  geom_ribbon(
    data = subset(df, distribution == "Mean 4" & x >= -1.96+4 & x <= 1.96+4),
    aes(x = x, ymin = 0, ymax = density),
    alpha = 0.1,
    inherit.aes = FALSE,
    fill = "red"
  ) +
  scale_color_manual(values = c("Mean 0" = "blue", "Mean 4" = "red")) +
  scale_fill_manual(values = c("Mean 0" = "blue", "Mean 4" = "red")) +
  scale_x_continuous(
    breaks = c(-4, -2, 0, 2, 4, 6, 8),
    labels = c(
      "-4", "-2",
      expression(mu[B]),
      "2",
      expression(mu[R]),
      "6", "8"
    )
  ) +
  labs(
    title = "Two Normal Distributions with Shaded Areas",
    x = NULL,
    y = NULL,
    color = "Distribution"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
    axis.text.y = element_blank(),
    axis.text.x = element_text(size = 14)
```

```
  ) +
  geom_vline(xintercept = 0, color = "blue", linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = 4, color = "red", linetype = "dashed", linewidth = 1)
```

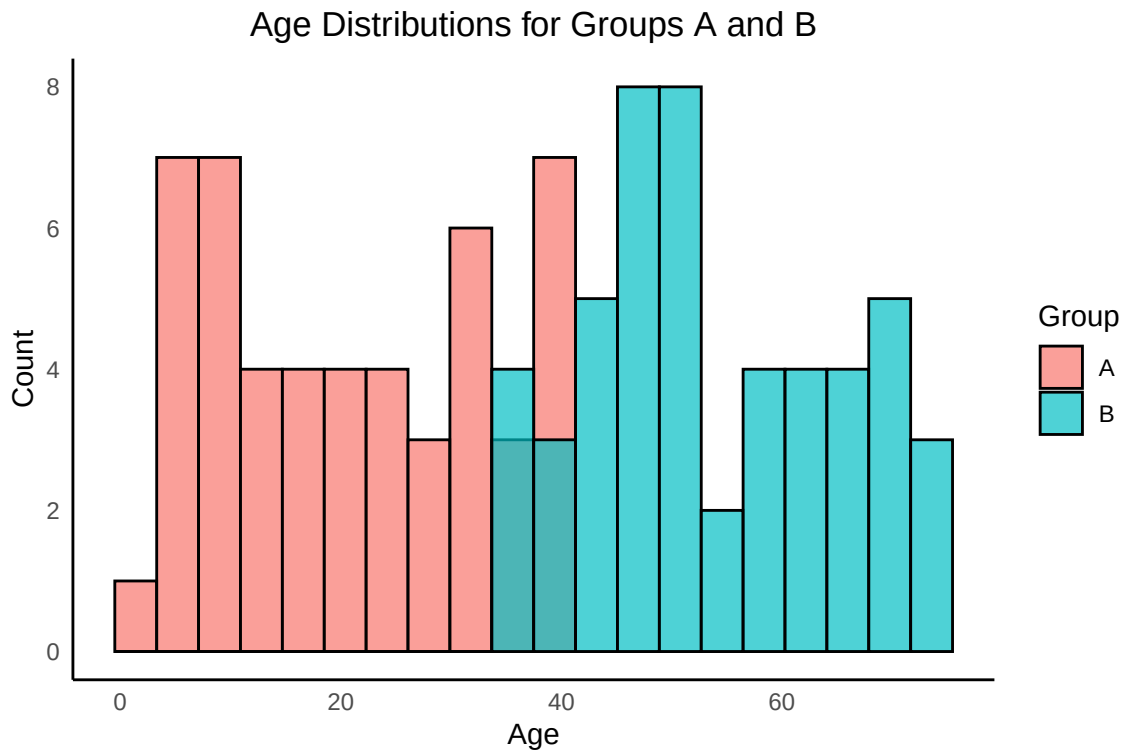


1.3 Plotting histograms of empirical data

Example: age distribution of two samples A and B.

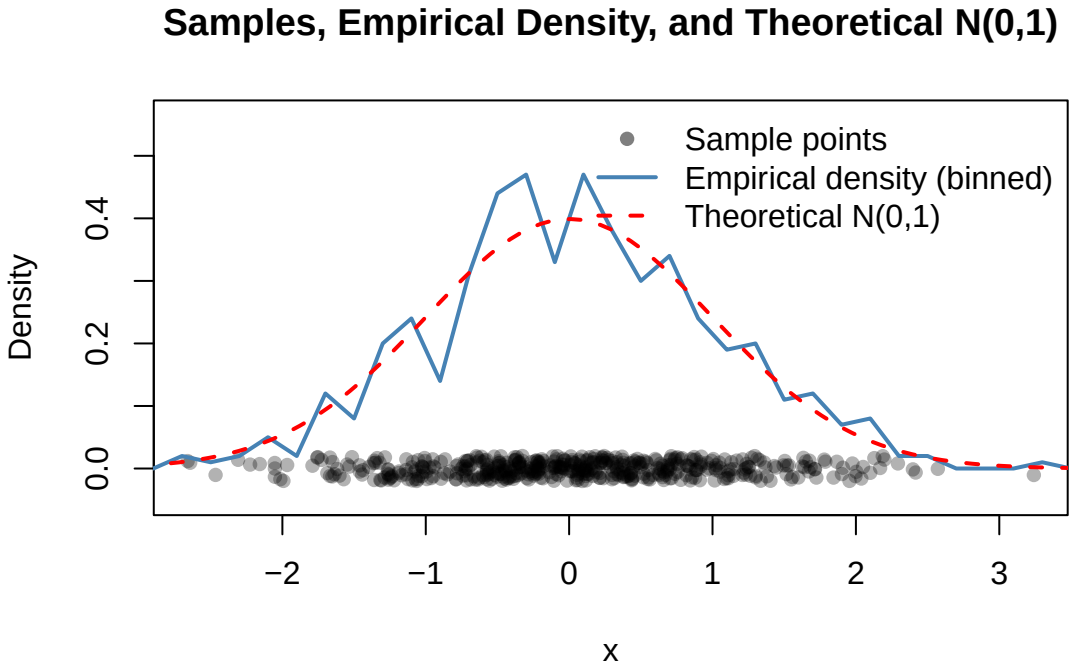
```
set.seed(2)  # for reproducibility

n <- 50
age_A <- runif(n, min = 1, max = 40)
age_B <- runif(n, min = 35, max = 74)

df <- data.frame(
  age = c(age_A, age_B),
  group = rep(c("A", "B"), each = n)
)

library(ggplot2)

ggplot(df, aes(x = age, fill = group)) +
  geom_histogram(position = "identity", alpha = 0.7, bins = 20, color = "black") +
  labs(
    title = "Age Distributions for Groups A and B",
    x = "Age",
    y = "Count",
    fill = "Group"
  ) +
  theme_minimal() +
```

```
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black")
  )
```




```
set.seed(123)

# 1. Sample data
N <- 500
x <- rnorm(N, mean = 0, sd = 1)

# Small vertical jitter so points don't overlap
y_jitter <- runif(N, min = -0.02, max = 0.02)

# 2. Empirical density via bins
dx <- 0.2
breaks <- seq(
  from = floor(min(x)),
  to   = ceiling(max(x)),
  by   = dx
)

h <- hist(x, breaks = breaks, plot = FALSE)

emp_density <- h$counts / (N * dx)
mids <- h$mids

# 3. Plot setup
```

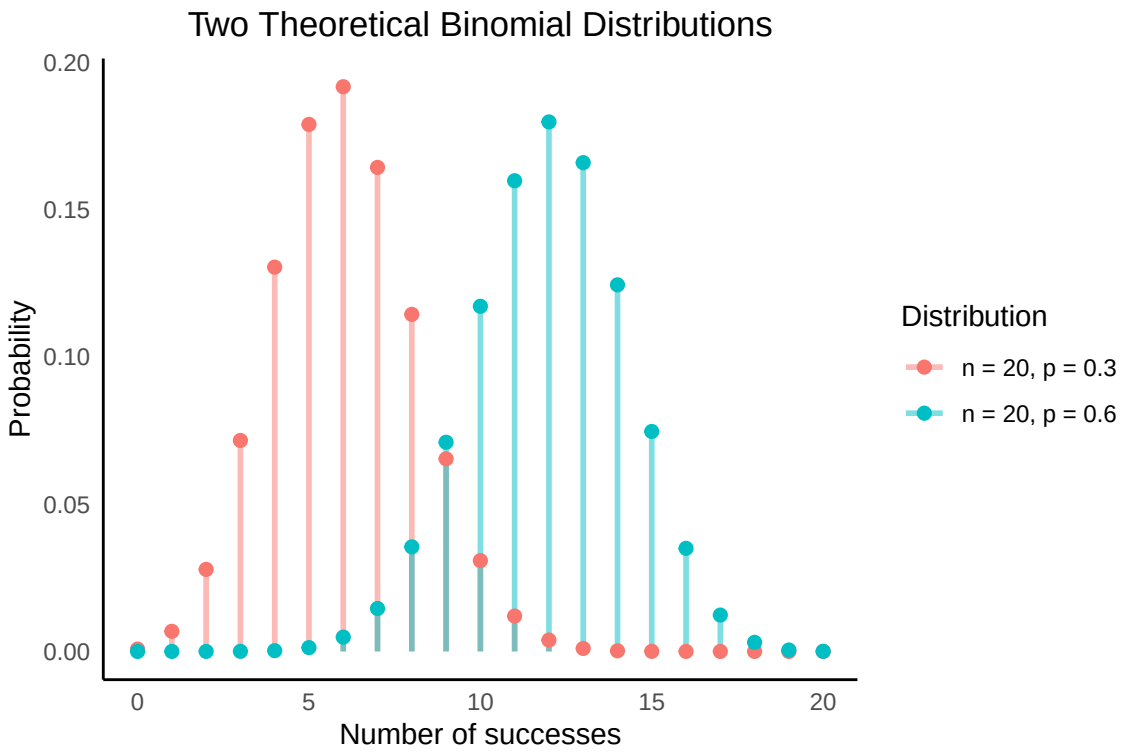
```r
plot(
  x, y_jitter,
  pch = 16,
  col = rgb(0, 0, 0, 0.3),
  xlab = "x",
  ylab = "Density",
  ylim = c(-0.05, max(emp_density) * 1.2),
  main = "Samples, Empirical Density, and Theoretical N(0,1)"
)

# 4. Empirical density as a solid line
lines(
  mids, emp_density,
  col = "steelblue",
  lwd = 2
)

# 5. Theoretical density
curve(
  dnorm(x, mean = 0, sd = 1),
  from = min(breaks),
  to   = max(breaks),
  add = TRUE,
  col = "red",
  lwd = 2,
  lty = 2
)

# 6. Legend
legend(
  "topright",
  legend = c(
    "Sample points",
    "Empirical density (binned)",
    "Theoretical N(0,1)"
  ),
  col = c(
    rgb(0, 0, 0, 0.5),
    "steelblue",
    "red"
  ),
  lwd = c(NA, 2, 2),
  lty = c(NA, 1, 2),
  pch = c(16, NA, NA),
  bty = "n"
)
```



1.4 Binomial distribution

```
x <- 0:20

df <- data.frame(
    x = rep(x, 2),
    probability = c(
    dbinom(x, size = 20, prob = 0.3),
    dbinom(x, size = 20, prob = 0.6)
  ),
  distribution = factor(rep(
    c("n = 20, p = 0.3", "n = 20, p = 0.6"),
    each = length(x)
  ))
)

library(ggplot2)


ggplot(df, aes(x = x, y = probability, color = distribution)) +
  geom_segment(
  aes(xend = x, y = 0, yend = probability),
    linewidth = 1,
    alpha = 0.5
  ) +
  geom_point(size = 2) +
  labs(
    title = "Two Theoretical Binomial Distributions",
    x = "Number of successes",
    y = "Probability",
    color = "Distribution"
  ) +
```

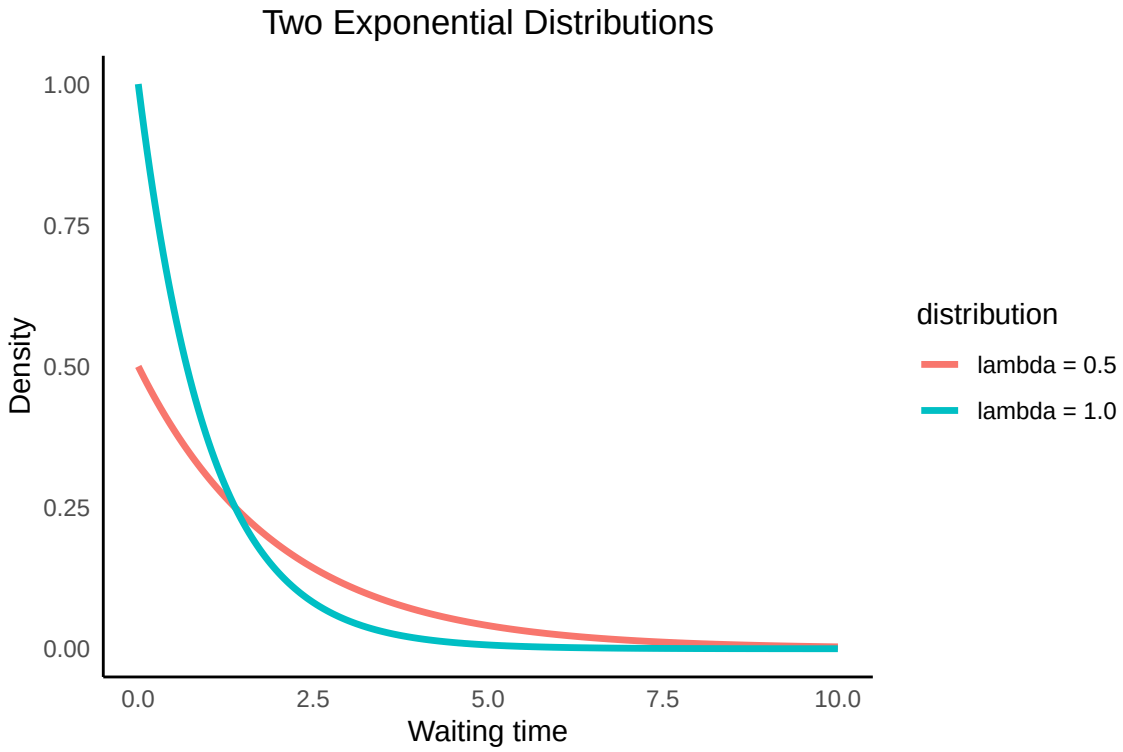
```
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```



1.5 Exponential distribution

```
library(ggplot2)

# x range (waiting time)
x <- seq(0, 10, length.out = 1000)

# Data frame with two exponential PDFs
df <- data.frame(
  x = rep(x, 2),
  density = c(
    dexp(x, rate = 0.5),
    dexp(x, rate = 1.0)
  ),
  distribution = factor(rep(
    c("lambda = 0.5", "lambda = 1.0"),
    each = length(x)
  ))
)

ggplot(df, aes(x = x, y = density, color = distribution)) +
```

```
  geom_line(linewidth = 1.2) +
  labs(
    title = "Two Exponential Distributions",
    x = "Waiting time",
    y = "Density"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```



2. The central limit theorem

In short, the central limit theorem implies that the sample mean is normally distributed. Let's do an empirical demonstration of this.

2.1 Example: rolling dice

First example is rolling dice. We will generate a random sample of 10 dice throws. And compute the average and standard deviation of the sample.

Then we will repeat the procedure and collect more samples of 10 dice throws. Finally we will plot the distribution of the average values that we get from each sample using a histogram. We will see that the distribution is going to be approximately normal.

We assume a fair die with numbers 1-6.

```r
set.seed(1) # for reproducibility

library(ggplot2)

# Simulate 10 dice throws (values 1 to 6)
throws <- sample(1:6, size = 10, replace = TRUE)

# Calculate sample mean and SD
mean_throws <- mean(throws)
sd_throws <- sd(throws)

# Create a data frame for plotting
df <- data.frame(value = throws)

# Plot histogram of throws with mean and SD lines
ggplot(df, aes(x = value)) +
  geom_histogram(binwidth = 1, boundary = 0.5, fill = "skyblue", color = "black") +
  geom_vline(xintercept = mean_throws, color = "red", size = 1, linetype = "solid") +
  geom_vline(xintercept = mean_throws + sd_throws, color = "red", size = 1, linetype = "dashed") +
  geom_vline(xintercept = mean_throws - sd_throws, color = "red", size = 1, linetype = "dashed") +
  labs(
    title = "Distribution of 10 Dice Throws",
    subtitle = paste0("Mean = ", round(mean_throws, 2), ", SD = ", round(sd_throws, 2)),
    x = "Dice Value",
    y = "Count"
  ) +
  scale_x_continuous(breaks = 1:6) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```

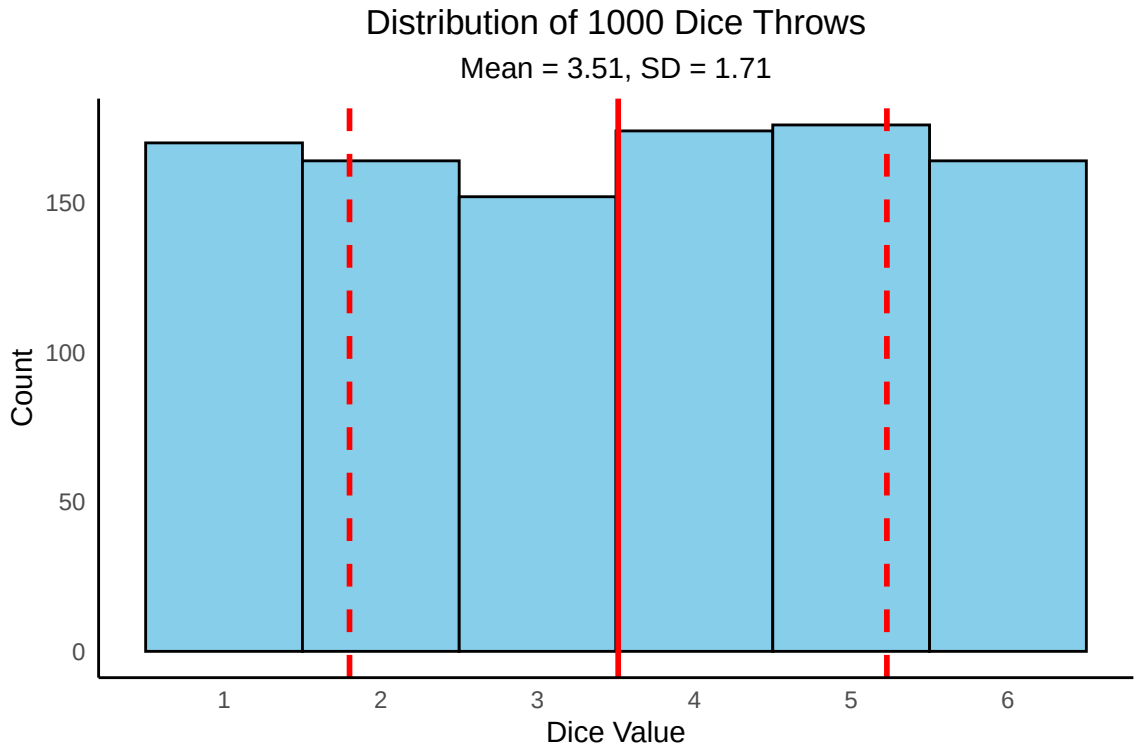
```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```


```r
set.seed(1) # for reproducibility

library(ggplot2)

# Simulate 100 dice throws (values 1 to 6)
throws <- sample(1:6, size = 100, replace = TRUE)

# Calculate sample mean and SD
mean_throws <- mean(throws)
sd_throws <- sd(throws)

# Create a data frame for plotting
df <- data.frame(value = throws)

# Plot histogram of throws with mean and SD lines
ggplot(df, aes(x = value)) +
  geom_histogram(binwidth = 1, boundary = 0.5, fill = "skyblue", color = "black") +
  geom_vline(xintercept = mean_throws, color = "red", size = 1, linetype = "solid") +
  geom_vline(xintercept = mean_throws + sd_throws, color = "red", size = 1, linetype = "dashed") +
  geom_vline(xintercept = mean_throws - sd_throws, color = "red", size = 1, linetype = "dashed") +
  labs(
    title = "Distribution of 100 Dice Throws",
    subtitle = paste0("Mean = ", round(mean_throws, 2), ", SD = ", round(sd_throws, 2)),
    x = "Dice Value",
    y = "Count"
  ) +
  scale_x_continuous(breaks = 1:6) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
```

```
    plot.subtitle = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```


```
set.seed(1) # for reproducibility

library(ggplot2)

# Simulate 1000 dice throws (values 1 to 6)
throws <- sample(1:6, size = 1000, replace = TRUE)

# Calculate sample mean and SD
mean_throws <- mean(throws)
sd_throws <- sd(throws)

# Create a data frame for plotting
df <- data.frame(value = throws)

# Plot histogram of throws with mean and SD lines
ggplot(df, aes(x = value)) +
  geom_histogram(binwidth = 1, boundary = 0.5, fill = "skyblue", color = "black") +
  geom_vline(xintercept = mean_throws, color = "red", size = 1, linetype = "solid") +
  geom_vline(xintercept = mean_throws + sd_throws, color = "red", size = 1, linetype = "dashed") +
  geom_vline(xintercept = mean_throws - sd_throws, color = "red", size = 1, linetype = "dashed") +
  labs(
    title = "Distribution of 1000 Dice Throws",
    subtitle = paste0("Mean = ", round(mean_throws, 2), ", SD = ", round(sd_throws, 2)),
    x = "Dice Value",
```

```
    y = "Count"
  ) +
  scale_x_continuous(breaks = 1:6) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```




```
set.seed(12)

n <- 10
B_vals <- c(50)

df <- data.frame()

for (B in B_vals) {
  sample_means <- numeric(B)

  for (i in 1:B) {
    x <- sample(1:6, size = n, replace = TRUE)
    sample_means[i] <- mean(x)
  }

  df <- rbind(
    df,
    data.frame(
```

```
      value = sample_means,
      experiments = factor(B)
    )
  )
}

library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

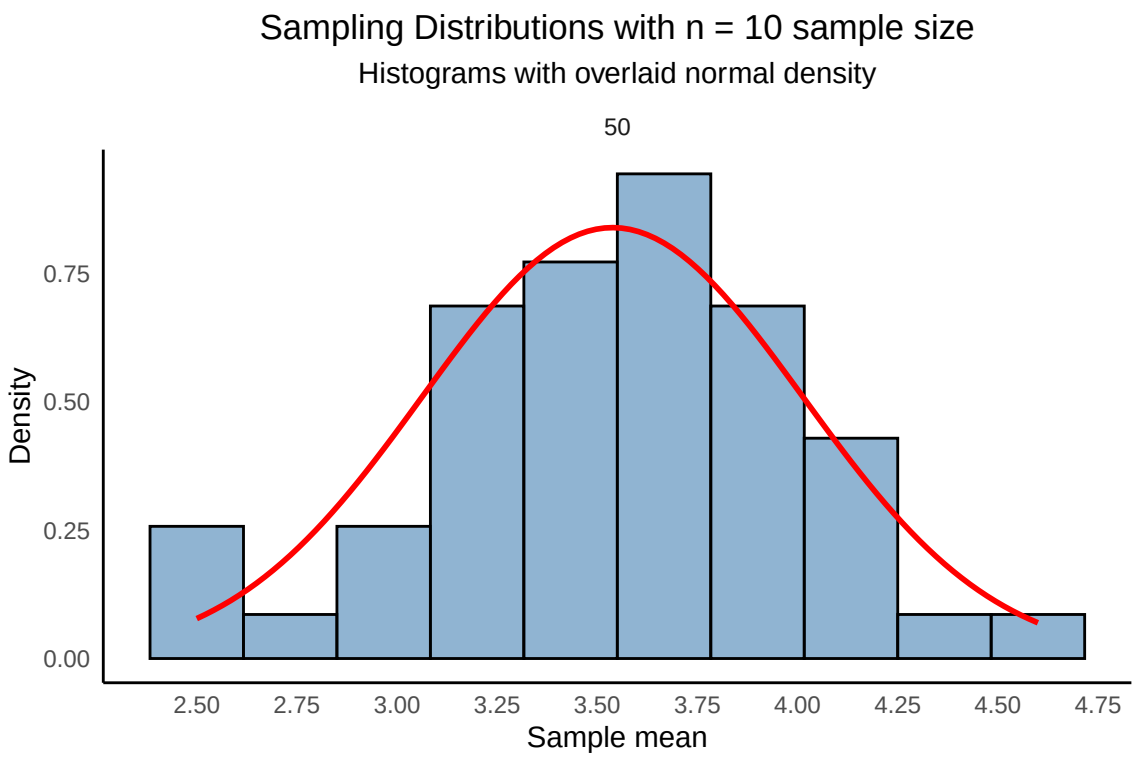
```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
stats <- df %>%
  group_by(experiments) %>%
  summarise(
    mean_val = mean(value),
    sd_val = sd(value)
  )

# Create a grid for normal curves
normal_curves <- stats %>%
  rowwise() %>%
  do({
    data.frame(
      experiments = .$experiments,
      x = seq(min(df$value), max(df$value), length.out = 100),
      y = dnorm(
        seq(min(df$value),
            max(df$value),
            length.out = 100),
        mean = .$mean_val,
        sd = .$sd_val)
    )
  }) %>%
  ungroup()

#experiments <- as.numeric(as.character(df$experiments))

library(ggplot2)

ggplot(df, aes(x = value)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 10,
    fill = "steelblue",
    color = "black",
    alpha = 0.6
```

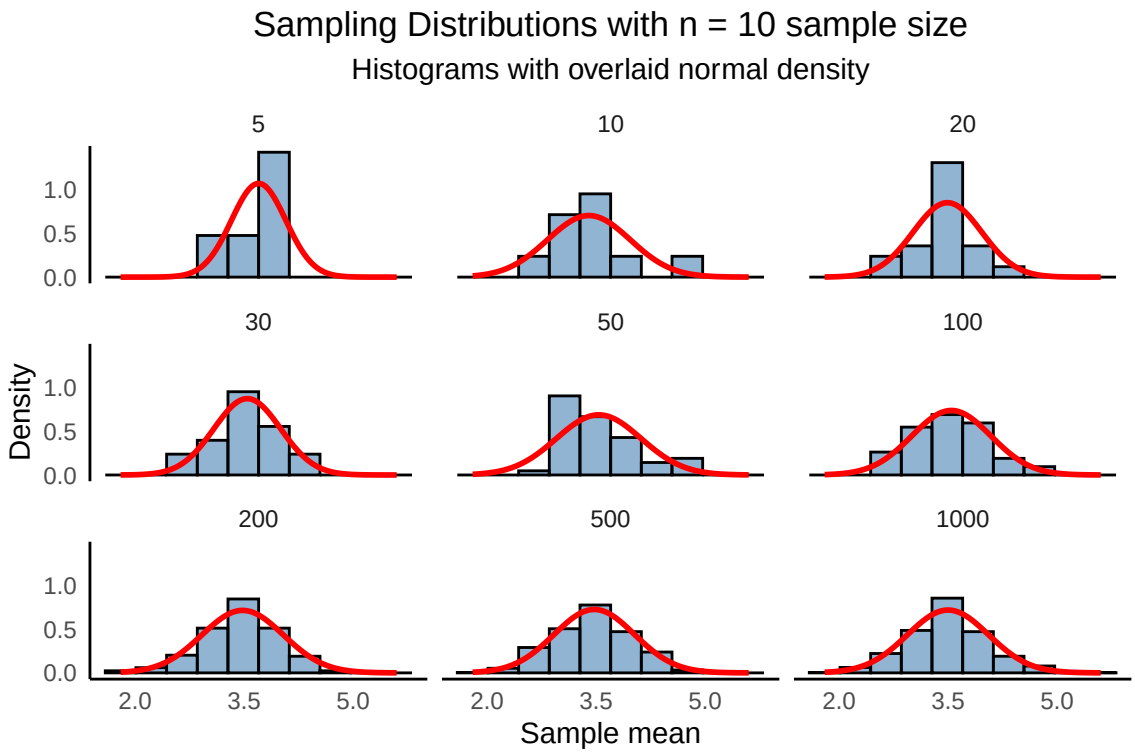
```
  ) +
  geom_line(
    data = normal_curves,
    aes(x = x, y = y),
    color = "red",
    size = 1
  ) +
  facet_wrap(~ experiments, ncol = 3) +
  labs(
    title = "Sampling Distributions with n = 10 sample size",
    subtitle = "Histograms with overlaid normal density",
    x = "Sample mean",
    y = "Density"
  ) +
  scale_x_continuous(breaks = seq(2, 5, by = 0.25)) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```




```
set.seed(12)

n <- 10
B_vals <- c(5, 10, 20, 30, 50, 100, 200, 500, 1000)

df <- data.frame()
```

```r
for (B in B_vals) {
  sample_means <- numeric(B)

  for (i in 1:B) {
    x <- sample(1:6, size = n, replace = TRUE)
    sample_means[i] <- mean(x)
  }

  df <- rbind(
    df,
    data.frame(
      value = sample_means,
      experiments = factor(B)
    )
  )
}

library(dplyr)

stats <- df %>%
  group_by(experiments) %>%
  summarise(
    mean_val = mean(value),
    sd_val = sd(value)
  )

# Create a grid for normal curves
normal_curves <- stats %>%
  rowwise() %>%
  do({
    data.frame(
      experiments = .$experiments,
      x = seq(min(df$value), max(df$value), length.out = 100),
      y = dnorm(
        seq(min(df$value),
            max(df$value),
            length.out = 100),
        mean = .$mean_val,
        sd = .$sd_val)
    )
  }) %>%
  ungroup()

#experiments <- as.numeric(as.character(df$experiments))

library(ggplot2)

ggplot(df, aes(x = value)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 10,
    fill = "steelblue",
    color = "black",
```

```
    alpha = 0.6
  ) +
  geom_line(
    data = normal_curves,
    aes(x = x, y = y),
    color = "red",
    size = 1
  ) +
  facet_wrap(~ experiments, ncol = 3) +
  labs(
    title = "Sampling Distributions with n = 10 sample size",
    subtitle = "Histograms with overlaid normal density",
    x = "Sample mean",
    y = "Density"
  ) +
  scale_x_continuous(breaks = seq(2, 5, by = 1.5)) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank(),
    axis.line = element_line(color = "black"),
  )
```



So based on this evolution of the distribution, the most common value of the average seems to be around 3.5. Comparing the empirical value of 3.5 with the theoretical expected value of a 6-sided die, where each side has

an equal probability of 1/6:

$$\mathbb{E}[X] = \sum_{k=1}^{6} k \cdot P(X = k) = \frac{(1+2+3+4+5+6)}{6} = \frac{21}{6} = 3.5$$

The shape of the distribution is also approaching a more normal distribution as the number of repeated experiments increase.

3. Survival analysis

Survival analysis is about analysing the expected time duration until one event occurs. A common example is time until death for a biological organism.

3.1 Survival function

The survival function is the primary object of interest. It is defined as:

$$S(t) = P(T > t) \tag{3.1}$$

The survival function is non-increasing:

$$S(u) \leq S(t) \ , \text{ for } u \geq t \tag{3.2}$$

This reflects the notion that survival to a later age is possible only if all younger ages are attained.

3.2 Lifetime distribution function and event density

The lifetime distribution function is the complement of the survival function:

$$F(t) = P(T \leq t) = 1 - S(t) \tag{3.3}$$

The event density is the rate of the lifetime distribution function:

$$f(t) = \frac{dF(t)}{dt} \tag{3.4}$$

The survival function can then be expressed in terms of a probability density function and a probability distribution:

$$S(t) = P(T > t) = \int_t^\infty f(u)\ du = 1 - F(t) \tag{3.5}$$

The survival event density can also be defined as:

$$s(t) = \frac{dS(t)}{dt} = -f(t) \tag{3.6}$$

3.3 Kaplan-Meier curves

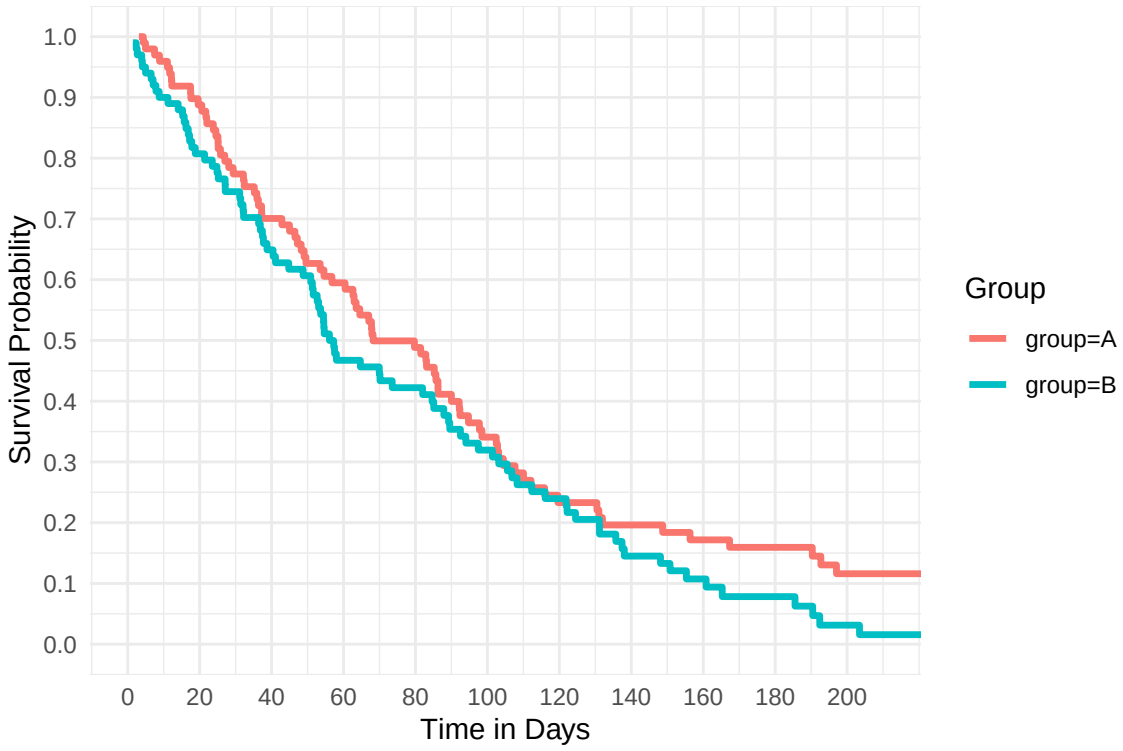
A Kaplan-Meier estimator is used to empirically estimate the survival function $S(t)$ (the probability that life is longer than t):

$$\hat{S}(t) = \prod_{i:\ t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \tag{3.7}$$

where t_i is a time where at least one event has happened, d_i is the number of events that happened at time t_i, and n_i are the invididuals that have survived (or been censored) up to time t_i.

Next a visual example:

```r
library(survival)
library(ggplot2)

set.seed(1)

# Simulate data: two groups, 100 individuals each
n <- 100
data <- data.frame(
  time = c(rexp(n, rate = 0.012), rexp(n, rate = 0.014)),  # exponential survival times
  status = c(rbinom(n, 1, 0.9), rbinom(n, 1, 0.9)),    # 100% event observed
  group = rep(c("A", "B"), each = n)
)

# Fit Kaplan-Meier
fit <- survfit(Surv(time, status) ~ group, data = data)

# Extract survival data into a data frame for ggplot
km_data <- do.call(rbind, lapply(seq_along(fit$strata), function(i) {
  idx <- sum(fit$strata[seq_len(i - 1)]) + 1L:fit$strata[i]
  data.frame(
    time = fit$time[idx],
    surv = fit$surv[idx],
    upper = fit$upper[idx],
    lower = fit$lower[idx],
    strata = rep(names(fit$strata)[i], fit$strata[i])
  )
}))

# Plot KM curves with confidence ribbons
ggplot(km_data, aes(x = time, y = surv, color = strata, fill = strata)) +
  geom_step(linewidth = 1.2) +
  labs(x = "Time in Days", y = "Survival Probability", color = "Group", fill = "Group") +
  coord_cartesian(xlim = c(0, 210), ylim = c(1.0, 0.0)) +
  scale_x_continuous(breaks = seq(0, 210, by = 20)) +    # ticks every 10 units
  scale_y_continuous(breaks = seq(0.0, 1.0, by = 0.1)) +     # ticks every 0.1
  theme_minimal()
```



4. Comparing two groups

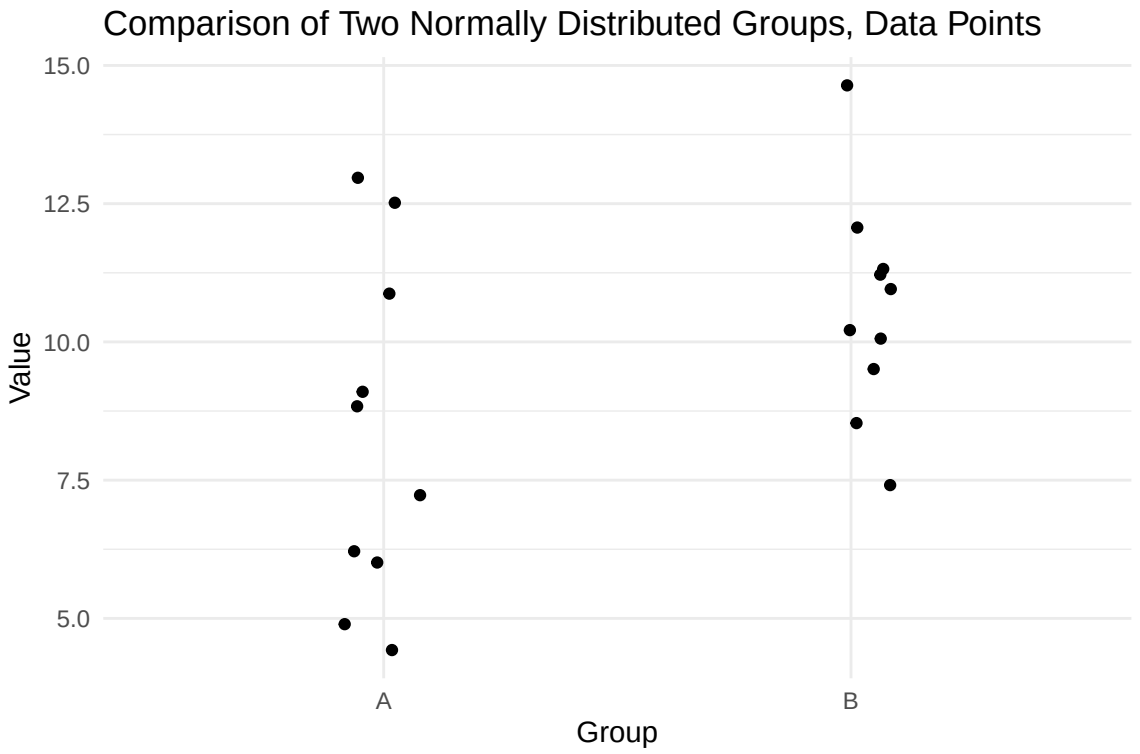
Measuring e.g. a continuous variable in two groups and visualising the measurements in each group.

```
set.seed(11)

# Sample size in both groups
n <- 10

A <- rnorm(n, mean = 9, sd = 3)
B <- rnorm(n, mean = 12, sd = 3)

df <- data.frame(
  value = c(A, B),
  group = factor(rep(c("A", "B"), each = n))
)
```

4.1 Raw data points

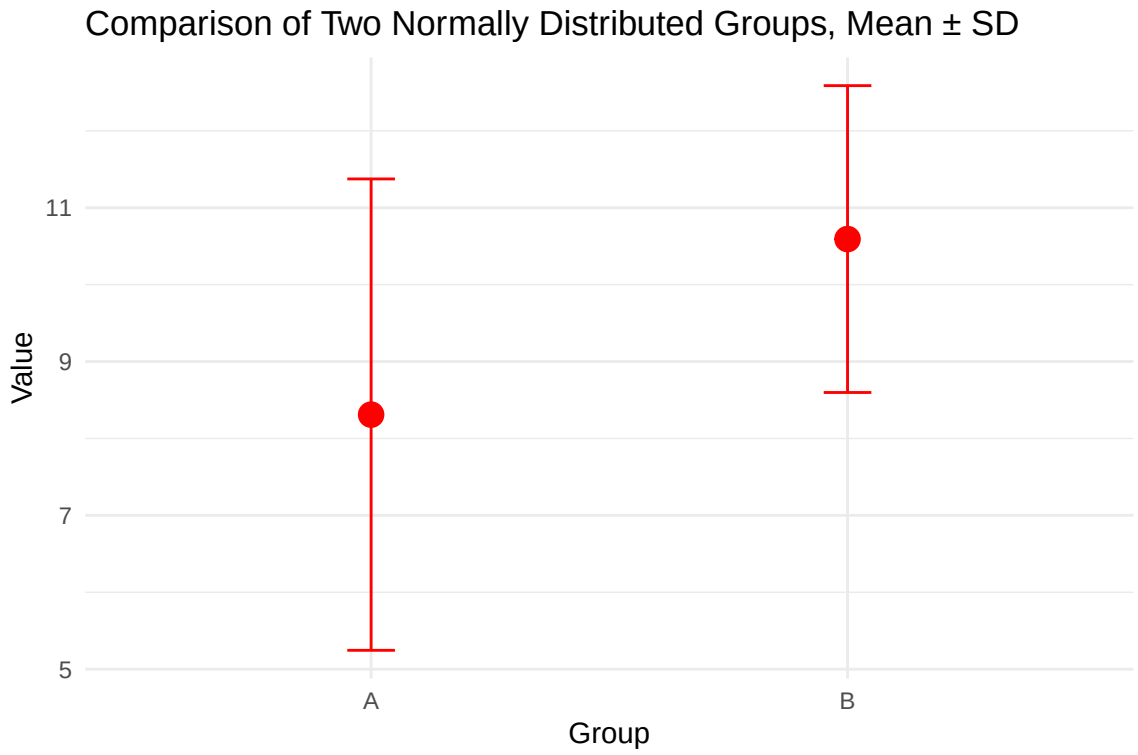
```
library(ggplot2)


# Data points
ggplot(df, aes(x = group, y = value)) +
  geom_jitter(width = 0.1, alpha = 1) +
  labs(
    title = "Comparison of Two Normally Distributed Groups, Data Points",
    x = "Group",
    y = "Value"
```

```
  ) +
  theme_minimal()
```



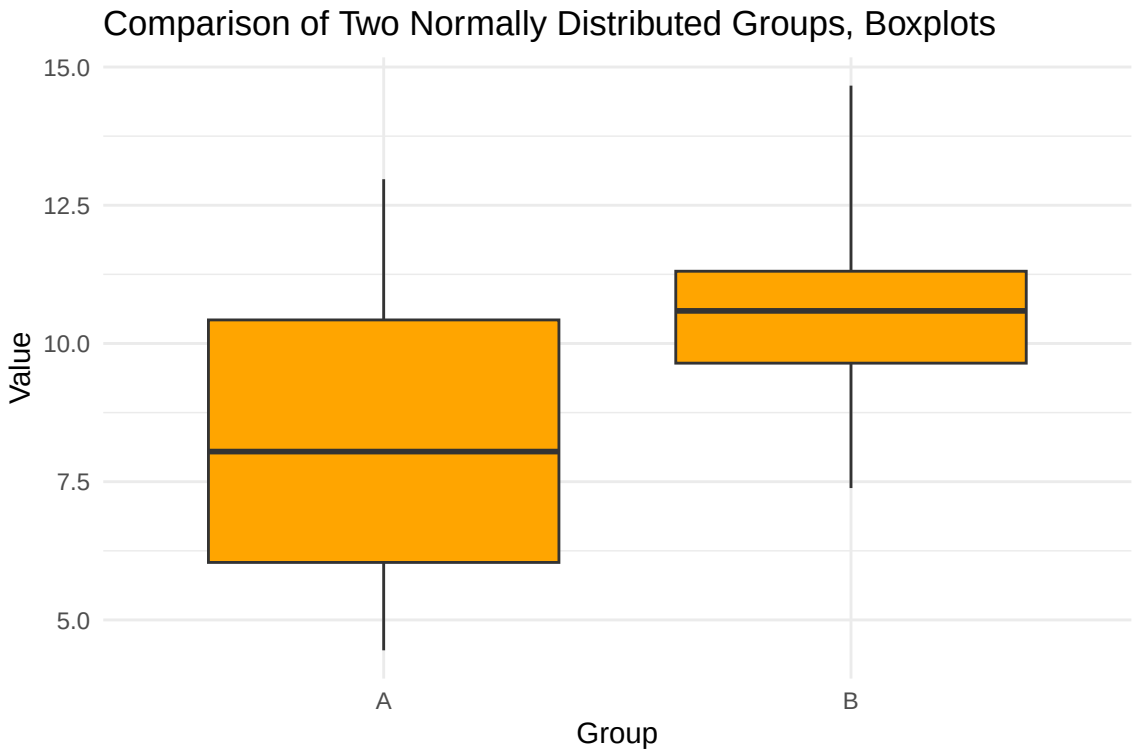
4.2 Mean ± SD

```
# Mean ± 1 SD
ggplot(df, aes(x = group, y = value)) +
  stat_summary(
    fun = mean,
    geom = "point",
    size = 4,
    color = "red"
  ) +
  stat_summary(
    fun.data = mean_sdl,
    fun.args = list(mult = 1),
    geom = "errorbar",
    width = 0.1,
    color = "red"
  ) +
  labs(
    title = "Comparison of Two Normally Distributed Groups, Mean ± SD",
    x = "Group",
    y = "Value"
  ) +
  theme_minimal()
```



4.3 Box plot with median, 25%, 75% and min/max values.

```r
# Boxplot
ggplot(df, aes(x = group, y = value)) +
  geom_boxplot(fill = "orange", size = 0.5, outlier.shape = NA, coef = Inf) +
  labs(
    title = "Comparison of Two Normally Distributed Groups, Boxplots",
    x = "Group",
    y = "Value"
  ) +
  theme_minimal()
```



5. Variations and outliers

Different contexts of variations in data sets, and how outliers can affect average values.

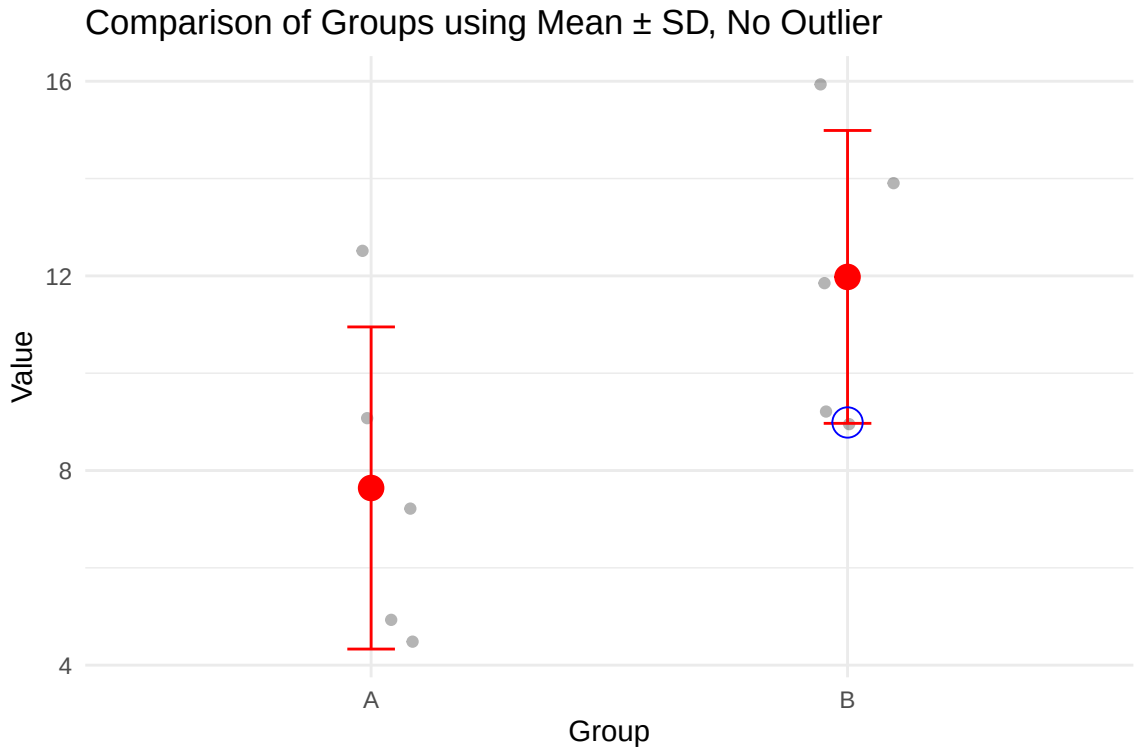
5.1 Outliers and group means

Taking the previous example of two groups. Plotting both the individual data points, and the mean ± SD.

Let's see how one outlier can affect the group mean. We will mark one point in group B data set and show the difference of the regularly generated point, versus if we increase the magnitude of it to make an arbitrary outlier.

```
set.seed(11)

# Sample size in both groups
n <- 5

A <- rnorm(n, mean = 9, sd = 3)
B <- rnorm(n, mean = 12, sd = 3)

df <- data.frame(
  value = c(A, B),
  group = factor(rep(c("A", "B"), each = n))
)

# Subset for the point to highlight (5th point in group B)
highlight_point <- df[df$group == "B", ][5, ]

# Plot data points and mean ± SD
ggplot(df, aes(x = group, y = value)) +
```

```
  geom_jitter(width = 0.1, alpha = 0.3) +
  stat_summary(
    fun = mean,
    geom = "point",
    size = 4,
    color = "red"
  ) +
  stat_summary(
    fun.data = mean_sdl,
    fun.args = list(mult = 1),
    geom = "errorbar",
    width = 0.1,
    color = "red"
  ) +
  geom_point(
    data = highlight_point,
    aes(x = group, y = value),
    color = "blue",
    size = 5,
    shape = 1
  ) +
  labs(
    title = "Comparison of Groups using Mean ± SD, No Outlier",
    x = "Group",
    y = "Value"
  ) +
  theme_minimal()
```




```
mean_A <- mean(df$value[df$group == "A"])
mean_B <- mean(df$value[df$group == "B"])
```

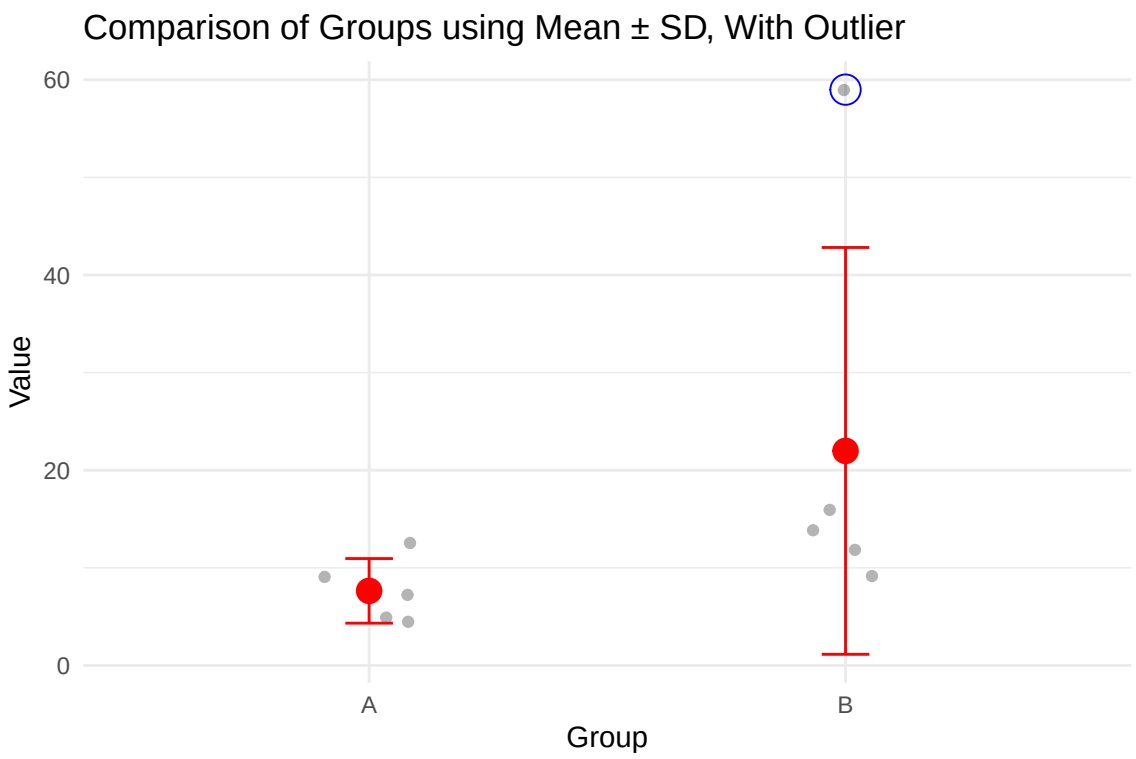
```r
mean_diff <- mean_B - mean_A

mean_diff
```

```
## [1] 4.33781
```

Similar to before, plotting mean ± SD of the two groups. The data point of interest is marked with a blue circle.

Now let's arbitrarily increase the magnitude of the blue data point to see its influence on the mean of group B.

```r
# Copy df to avoid altering original data accidentally
df_mod <- df

# Add 50 to the blue point's value
df_mod[df_mod$group == "B", ][5, ]$value <- df_mod[df_mod$group == "B", ][5, ]$value + 50

highlight_point <- df_mod[df_mod$group == "B", ][5, ]

# Plot data points and mean ± SD
ggplot(df_mod, aes(x = group, y = value)) +
  geom_jitter(width = 0.1, alpha = 0.3) +
  stat_summary(
    fun = mean,
    geom = "point",
    size = 4,
    color = "red"
  ) +
  stat_summary(
    fun.data = mean_sdl,
    fun.args = list(mult = 1),
    geom = "errorbar",
    width = 0.1,
    color = "red"
  ) +
  geom_point(
    data = highlight_point,
    aes(x = group, y = value),
    color = "blue",
    size = 5,
    shape = 1
  ) +
  labs(
    title = "Comparison of Groups using Mean ± SD, With Outlier",
    x = "Group",
    y = "Value"
  ) +
  theme_minimal()
```




```
mean_A <- mean(df_mod$value[df_mod$group == "A"])
mean_B <- mean(df_mod$value[df_mod$group == "B"])

mean_diff <- mean_B - mean_A

mean_diff
```

```
## [1] 14.33781
```

As can be seen, the outlier marked with a blue circle shifts the mean value upwards, consequently increasing the mean difference between the groups as well.

It also increases the standard deviation of the whole data set.

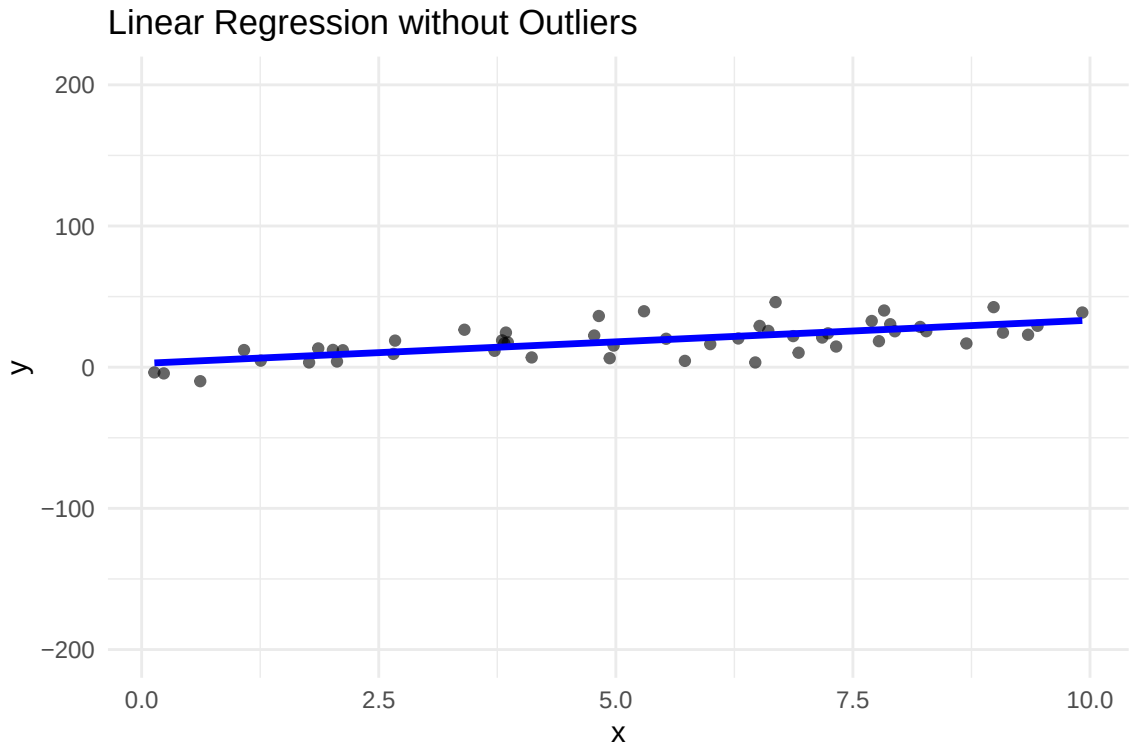
5.2 Outliers and linear regression

Linear regression is another example where outliers can arbitrarily inflate coefficients.

```
set.seed(1)

n <- 50
x <- runif(n, 0, 10)
y <- 2 + 3 * x + rnorm(n, 0, 10)  # y = 2 + 3x + noise

df <- data.frame(x = x, y = y)

model <- lm(y ~ x, data = df)

library(ggplot2)

ggplot(df, aes(x = x, y = y)) +
  geom_point(alpha = 0.6) +
```

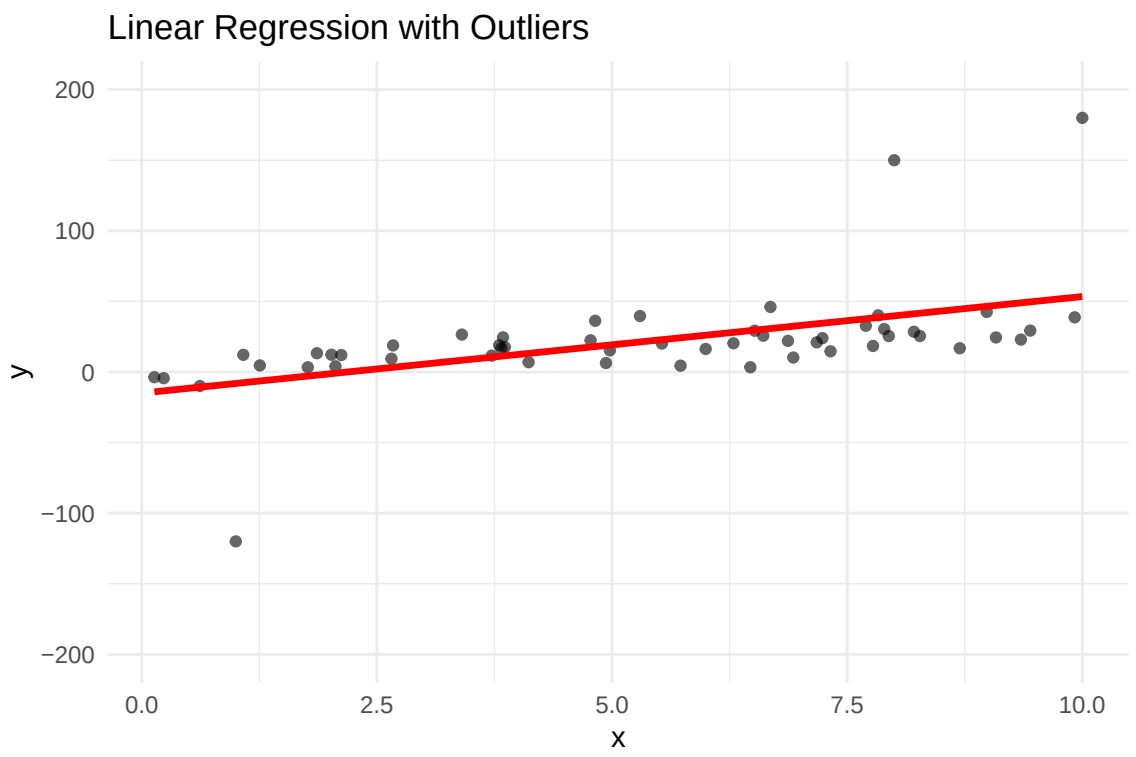
```
  geom_smooth(method = "lm", se = FALSE, color = "blue",linewidth = 1.2) +
  ylim(-200, 200) +
  labs(title = "Linear Regression without Outliers", x = "x", y = "y") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



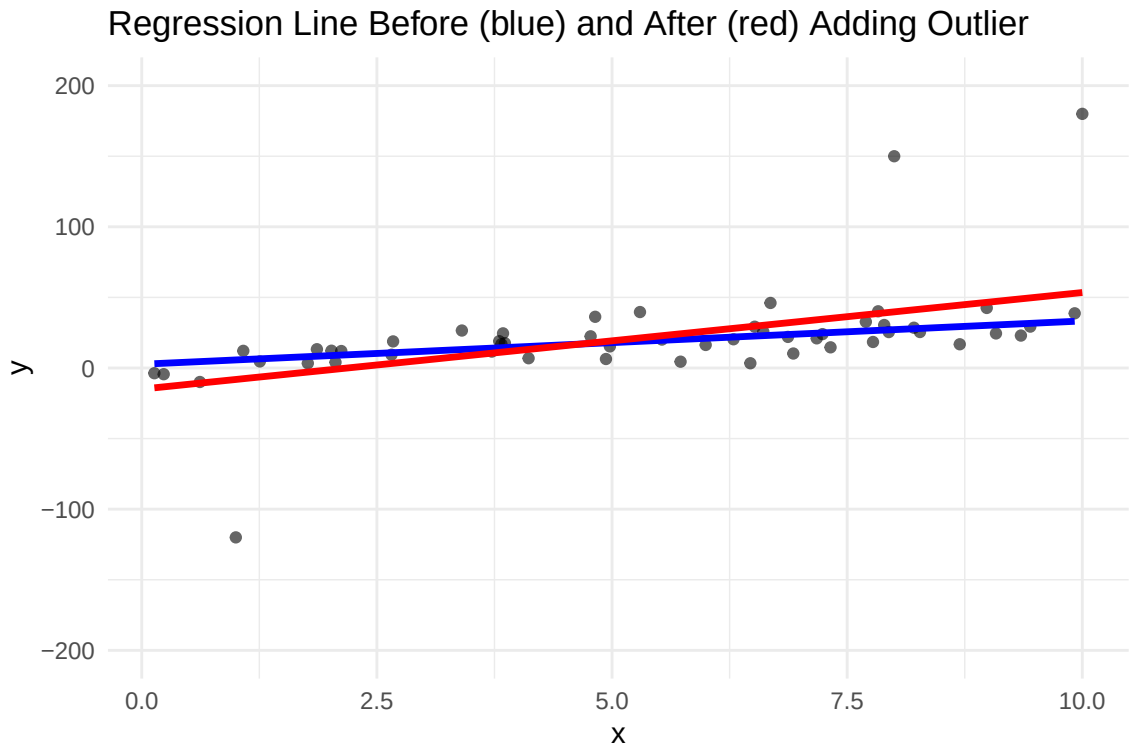

```
# Add outliers
df_outlier <- rbind(df, data.frame(x = 8, y = 150))  # extreme y value at x=8
df_outlier <- rbind(df_outlier, data.frame(x = 1, y = -120))  # extreme y value at x=1
df_outlier <- rbind(df_outlier, data.frame(x = 10, y = 180))  # extreme y value at x=10

# Fit model with outlier
model_outlier <- lm(y ~ x, data = df_outlier)

ggplot(df_outlier, aes(x = x, y = y)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE, color = "red", linewidth = 1.2) +
  ylim(-200, 200) +
  labs(title = "Linear Regression with Outliers", x = "x", y = "y") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
ggplot() +
  geom_point(data = df_outlier, aes(x = x, y = y), alpha = 0.6) +
  geom_smooth(data = df, aes(x = x, y = y), method = "lm", se = FALSE, color = "blue", linewidth = 1.2)
  geom_smooth(data = df_outlier, aes(x = x, y = y), method = "lm", se = FALSE, color = "red", linewidth
  ylim(-200, 200) +
  labs(title = "Regression Line Before (blue) and After (red) Adding Outlier",
       x = "x", y = "y") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```



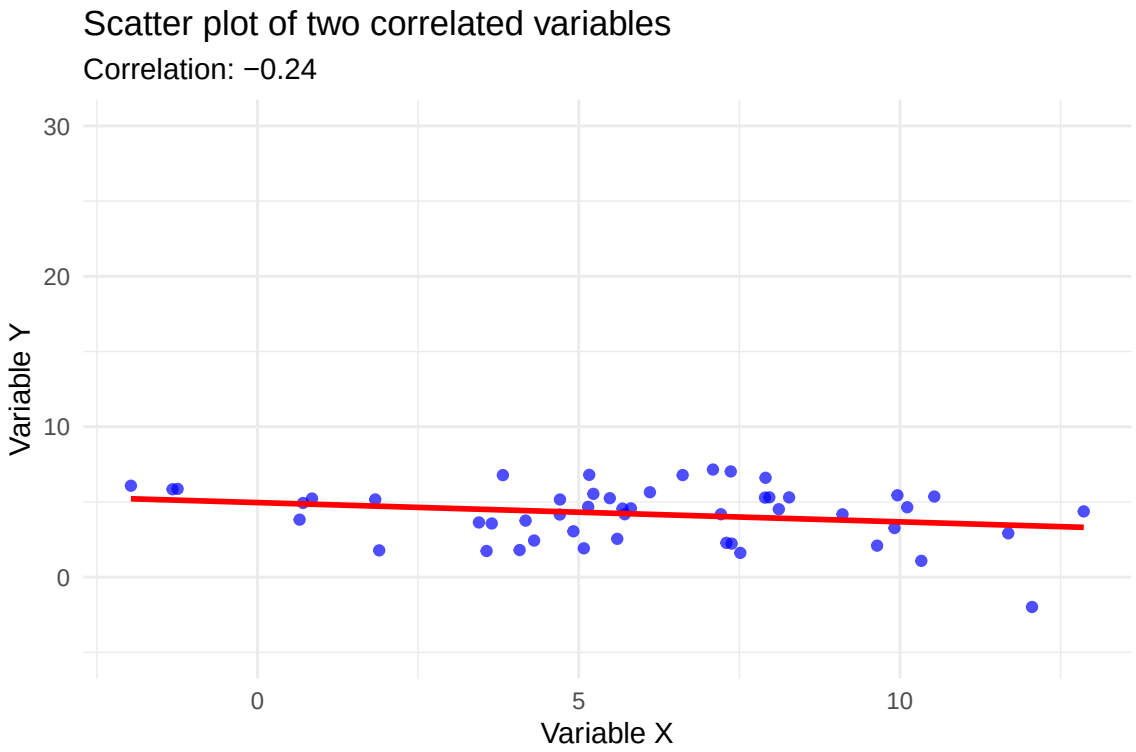
The red regression line is rotated towards the outliers, arbitrarily increasing the slope coefficient.

5.3 Outliers and linear correlation

Consequently, since outliers affect the linear slope of a regression, they also influence the linear correlation.

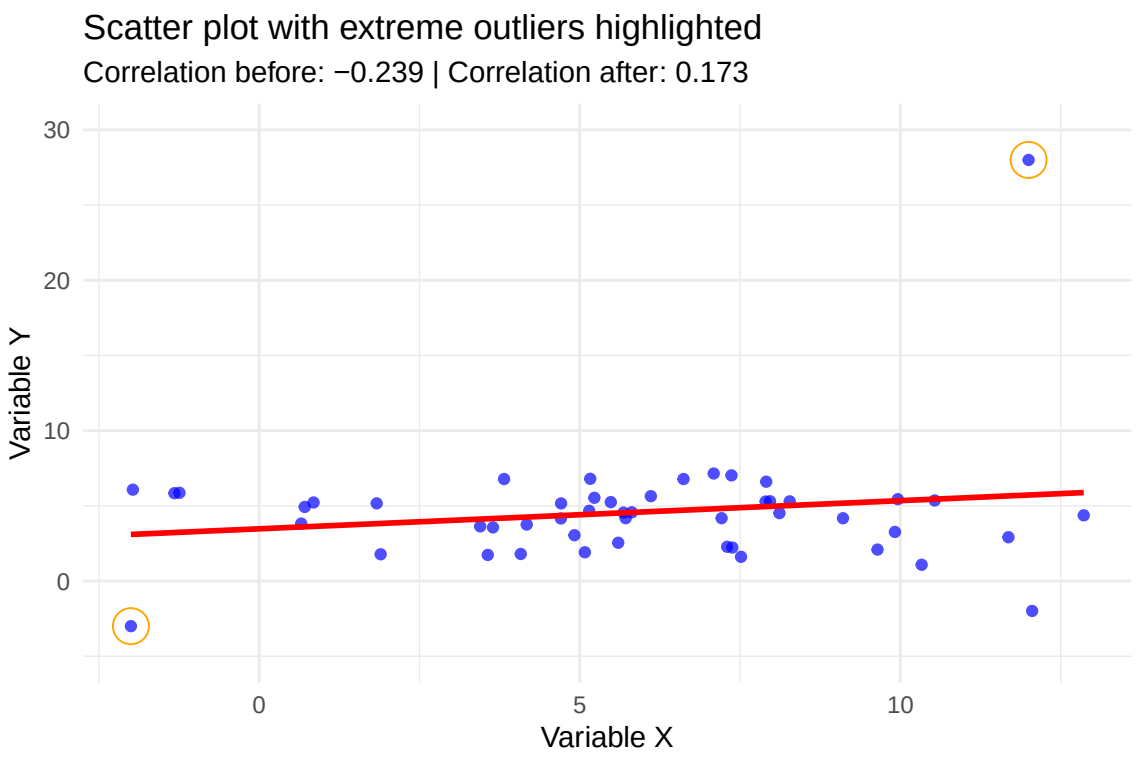
```r
set.seed(42)

n <- 50

# Generate two variables with correlation

x <- rnorm(n, mean = 6, sd = 3)
y <- rnorm(n, mean = 4, sd = 2)

df <- data.frame(x = x, y = y)

library(ggplot2)

ggplot(df, aes(x = x, y = y)) +
  geom_point(color = "blue", alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  ylim(-5,30) +
  labs(
    title = "Scatter plot of two correlated variables",
    subtitle = paste("Correlation:", round(cor(df$x, df$y), 2)),
    x = "Variable X",
    y = "Variable Y"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```r
# Modify points to extreme outliers
df_mod <- df
df_mod$x[1] <- 12
df_mod$y[1] <- 28
df_mod$x[2] <- -2
df_mod$y[2] <- -3

# Calculate correlation before and after
cor_before <- cor(df$x, df$y)
cor_after <- cor(df_mod$x, df_mod$y)

# Plot with highlighted point
ggplot(df_mod, aes(x = x, y = y)) +
  geom_point(color = "blue", alpha = 0.7) +
  geom_point(data = df_mod[1, ], aes(x = x, y = y), color = "orange", size = 6, shape = 1) +
  geom_point(data = df_mod[2, ], aes(x = x, y = y), color = "orange", size = 6, shape = 1) +
  geom_smooth(method = "lm", se = FALSE, color = "red", size = 1) +
  ylim(-5,30) +
  labs(
    title = "Scatter plot with extreme outliers highlighted",
    subtitle = paste0("Correlation before: ", round(cor_before, 3),
                      " | Correlation after: ", round(cor_after, 3)),
    x = "Variable X",
    y = "Variable Y"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



As seen, the outliers manage to invert the correlation value.